

Lesson 15: Modules Basics

Learning Objectives

- Understand what Terraform modules are and why they exist
- Create a local module with inputs and outputs
- Call a module from a root configuration
- Understand module structure conventions

1. What is a Module?

A **module** is a **self-contained collection of Terraform configuration** that manages a group of related resources. Every Terraform configuration is technically a module — the root module.

```
project/
├── main.tf           # Root module – calls child modules
├── provider.tf
├── variables.tf
├── outputs.tf
└── modules/
    └── s3-bucket/    # Child module
        ├── main.tf  # Resources this module creates
        ├── variables.tf # Inputs the module accepts
        └── outputs.tf # Values the module exposes
```

Why use modules?

Benefit	Description
Reusability	Write once, use across multiple environments and projects
Abstraction	Hide complexity behind a simple interface (inputs → outputs)
Consistency	Same configuration pattern for every bucket, queue, table
Testing	Test a module once, deploy everywhere
Team collaboration	Teams share modules via registries

2. Module Structure Conventions

A minimal module has three files:

```
modules/s3-bucket/
├── main.tf          # Required – resource definitions
```

```
|— variables.tf    # Optional – input variables
|— outputs.tf     # Optional – output values
```

The `main.tf` — what the module creates

```
# modules/s3-bucket/main.tf
resource "aws_s3_bucket" "this" {
  bucket = var.bucket_name
  tags   = var.tags
}

resource "aws_s3_bucket_versioning" "this" {
  bucket = aws_s3_bucket.this.id

  versioning_configuration {
    status = var.enable_versioning ? "Enabled" : "Suspended"
  }
}
```

The `variables.tf` — inputs the module accepts

```
# modules/s3-bucket/variables.tf
variable "bucket_name" {
  description = "Name of the S3 bucket"
  type        = string
}

variable "enable_versioning" {
  description = "Enable S3 bucket versioning"
  type        = bool
  default     = true
}

variable "tags" {
  description = "Tags to apply to the bucket"
  type        = map(string)
  default     = {}
}
```

The `outputs.tf` — values the module exposes

```
# modules/s3-bucket/outputs.tf
output "bucket_id" {
  description = "ID (name) of the bucket"
  value       = aws_s3_bucket.this.id
}
```

```
output "bucket_arn" {
  description = "ARN of the bucket"
  value      = aws_s3_bucket.this.arn
}

output "bucket_domain" {
  description = "Domain name of the bucket"
  value      = aws_s3_bucket.this.bucket_domain_name
}
```

3. Calling a Module

The **root module** calls a child module with the **module** block:

```
# main.tf (root)
module "storage" {
  source = "../modules/s3-bucket"

  bucket_name      = "my-app-data"
  enable_versioning = true
  tags = {
    Environment = "dev"
    ManagedBy   = "Terraform"
  }
}
```

Module Block Syntax

```
module "<local_name>" {
  source    = "<path or registry address>"
  <input_variable> = <value>
}
```

Argument	Description
source	Path to module (local path or registry URL)
version	Module version (for registry modules)
input variables	Any variables declared in the module's variables.tf

Referencing Module Outputs

```
output "bucket_arn" {
  value = module.storage.bucket_arn
}
```

```
}

# Use in another resource
resource "aws_sqs_queue" "notifications" {
  name = "notifications-queue"

  tags = {
    SourceBucket = module.storage.bucket_id
  }
}
```

Module outputs are accessed as `module.<module_name>.<output_name>`.

4. Calling a Module Multiple Times

You can call the same module multiple times with different inputs:

```
module "data_bucket" {
  source = "./modules/s3-bucket"

  bucket_name      = "app-data"
  enable_versioning = true
}

module "logs_bucket" {
  source = "./modules/s3-bucket"

  bucket_name      = "app-logs"
  enable_versioning = true
  tags = {
    Environment = "dev"
    Purpose     = "logging"
  }
}

module "backup_bucket" {
  source = "./modules/s3-bucket"

  bucket_name      = "app-backup"
  enable_versioning = false # override default
}
```

5. Complete Example

Module: `modules/sqs-queue/main.tf`

```
resource "aws_sqs_queue" "this" {
  name                    = var.queue_name
  delay_seconds           = var.delay_seconds
  message_retention_seconds = var.retention_seconds
  receive_wait_time_seconds = var.receive_wait_time
  visibility_timeout_seconds = var.visibility_timeout
  tags                   = var.tags
}
```

Module: `modules/sqs-queue/variables.tf`

```
variable "queue_name" {
  description = "Name of the SQS queue"
  type        = string
}

variable "delay_seconds" {
  description = "Delay in seconds"
  type        = number
  default     = 0
}

variable "retention_seconds" {
  description = "Message retention in seconds"
  type        = number
  default     = 345600
}

variable "receive_wait_time" {
  description = "Wait time for long polling"
  type        = number
  default     = 0
}

variable "visibility_timeout" {
  description = "Visibility timeout in seconds"
  type        = number
  default     = 30
}

variable "tags" {
  description = "Tags to apply"
  type        = map(string)
  default     = {}
}
```

Module: `modules/sqs-queue/outputs.tf`

```
output "queue_id" {
  value = aws_sqs_queue.this.id
}

output "queue_arn" {
  value = aws_sqs_queue.this.arn
}

output "queue_url" {
  value = aws_sqs_queue.this.url
}
```

Root: `main.tf`

```
provider "aws" {
  # ... LocalStack config
}

module "order_events" {
  source = "../modules/sqs-queue"

  queue_name      = "order-events"
  delay_seconds   = 5
  tags = {
    Environment = "dev"
  }
}

module "notification_queue" {
  source = "../modules/sqs-queue"

  queue_name      = "notifications"
  retention_seconds = 604800 # 7 days
}
```

6. Providers in Modules

By default, modules inherit the provider from the root configuration. You don't need to declare a provider inside a module unless you need a different configuration.

```
# Root – provider is defined once
provider "aws" {
  region = "us-east-1"
}

# Module – automatically uses root's provider
module "bucket" {
```

```
source = "./modules/s3-bucket"  
# No provider block needed here  
}
```

7. Key Takeaways

- A module is a directory with `.tf` files, typically `main.tf`, `variables.tf`, `outputs.tf`
- Modules are called with `module` blocks using `source` to point to the module path
- Module inputs are declared in `variables.tf` — required variables must be set by the caller
- Module outputs are accessed as `module.<name>.<output_name>`
- Same module can be called multiple times with different inputs
- Modules inherit the provider from the root configuration
- `source` can be a local path or a registry URL (e.g., `terraform-aws-modules/s3-bucket/aws`)

example and exercise repo url

<https://github.com/eyu-se/terraform-training-material-with-projects>

Trainer - Eyuel M.